



Документ виправлено

Всі пункти Checklist враховані. Готово до завантаження.

Download PDF

AGENT STUDIO

ProofFlow AI Technical Specification

AI-powered pre-launch documentation verification and risk intelligence platform for Web3 projects

Executive Summary

ProofFlow AI is a Python-based, LLM-powered verification platform that ingests Web3 project artifacts—whitepapers, GitHub repositories, smart contract ABIs, tokenomics data, and website content—and cross-references them to surface contradictions, missing security disclosures, and investor-risk signals before launch. The system delivers a structured risk scorecard (High / Medium / Low) with cited evidence and remediation guidance, targeting launchpads, auditors, and institutional investors as primary customers. The MVP is scoped for 10 weeks at a \$40–70k budget with a team of five.

Chain-agnostic: ingests ABIs from Ethereum, Base, Arbitrum, Polygon via Etherscan and Sourcify

Core stack: FastAPI + LangChain + GPT-4o (Claude fallback) + Pinecone + Next.js

Output: exportable PDF risk report with per-claim citations and confidence scores

Deployment model: cloud SaaS for launchpads; on-premise Llama 2 option for enterprise privacy (see Security Considerations for hardware and framework specifications)

Monetization: per-report credits, launchpad API subscriptions (\$500–5,000/month)

CORE VALUE PROPOSITION

ProofFlow AI closes the gap between zero pre-launch verification and a \$5k–50k manual audit by delivering an AI-generated risk report in minutes, not weeks—without requiring code integration from the project team.

Problem Statement

Web3 projects routinely publish fragmented and internally inconsistent documentation across whitepapers, smart contracts, GitHub repositories, and marketing websites. Auditors, launchpads, and investors currently reconcile these sources manually—a process that takes days, costs \$5k–50k, and creates a 2–4 week bottleneck that forces many projects to launch with unresolved discrepancies. No automated tool exists that cross-references all documentation sources using AI to detect contradictions before manual review begins.

Existing Tool	Coverage Gap
CertiK Skynet	Contract code only; no whitepaper-to-contract consistency check
Token Sniffer	Honeypot/scam detection; no documentation quality analysis
GoPlus Security API	Predefined on-chain signals; no LLM-driven contradiction detection
De.Fi Scanner	Smart contract vulnerability scanning; no cross-document consistency checking or whitepaper analysis
Dune Dashboards	Post-launch tokenomics visualization; manual, no automation
Manual Audit Firms	\$5k–50k+, 2–4 week wait; not scalable for launchpad volume

SEVERITY

High. The pre-audit bottleneck affects 10,000+ token launches per year across a \$50B+ annual launch volume, with regulatory pressure from SEC and CFTC accelerating demand for documented pre-launch due diligence.

Solution Architecture

ProofFlow AI is built as a five-layer pipeline: (1) multi-source document ingestion, (2) structured claim extraction via LLM prompt chains, (3) cross-source consistency checking using a deterministic rule engine plus RAG-grounded LLM reasoning, (4) risk scoring aggregation, and (5) report generation. An async task queue (Celery + Redis) decouples ingestion from inference, preventing API timeouts on large documents. All LLM responses are grounded in retrieved document excerpts via Pinecone vector search to reduce hallucination.

Ingestion: unstructured.io handles PDF, HTML, and Markdown; web3.py fetches ABIs from Etherscan/Sourcify; GitHub API (authenticated, 5,000 req/hr) pulls README and Solidity files

Extraction: structured JSON output enforced via Pydantic schemas—token supply, vesting schedules, team disclosures, audit references, contract permissions

Consistency engine: deterministic rules (e.g., supply sum $\neq 100\% \pm 0.1\%$) combined with LLM semantic contradiction detection across source pairs

Risk scoring: weighted aggregation of rule violations and LLM-flagged issues into High / Medium / Low scorecard with per-issue confidence scores

Report: Next.js-rendered PDF export with citations linking each flag to the originating document and line/page number

ARCHITECTURE DECISION

RAG (retrieval-augmented generation) over Pinecone is mandatory for MVP—every LLM claim must cite a retrieved excerpt, enabling confidence thresholds and human review triggers rather than relying on unconstrained generation.

Smart Contract Design

ProofFlow AI requires no on-chain smart contract execution for the MVP. The platform performs static analysis of externally sourced ABIs and Solidity source code retrieved from Etherscan, Sourcify, and GitHub. The smart contract specialist's role is domain analysis: identifying anti-patterns in ingested ABIs (owner-only minting, pausable without timelock, upgradeable proxy without governance delay) rather than deploying new contracts. Proxy pattern detection is implemented via 4-byte function selector lookup against the deployed bytecode using web3.py's `eth_getCode` call combined with known selector registries (e.g., `4byte.directory`); for contracts with verified source code, static analysis of inheritance and delegatecall patterns is performed directly on the Solidity AST. Minimal proxy detection follows ERC-1167 (Clones), matching the standard bytecode prefix (`0x363d3d373d3d3d363d73...`) in the retrieved bytecode. This analysis is limited to contracts with verified source on Etherscan or Sourcify; unverified bytecode-only contracts receive a flag noting reduced analysis confidence. A minimal on-chain credentialing contract may be introduced post-MVP to anchor report hashes for tamper-evident audit trails.

Analysis Target	Extraction Method	Risk Signals Detected
ERC-20 ABI (token contract)	web3.py + Etherscan API	Unrestricted mint, blacklist function, owner withdraw
ERC-721/1155 ABI	Sourcify fallback if Etherscan unverified	Centralized metadata URI, no royalty enforcement
Proxy pattern (ERC-1967, ERC-1822/UUPS, ERC-1167/Clones)	4-byte selector lookup + bytecode prefix matching via web3.py <code>eth_getCode</code> ; Solidity AST analysis for verified contracts	Upgrade without timelock, admin key single-sig, uninitialized implementation slot
Vesting contract ABI	GitHub Solidity parser	Cliff mismatch vs. whitepaper, owner-override release
Governance contract ABI	Etherscan verified source	Quorum below 5%, no timelock on execution

KEY CONSTRAINT

Unverified contracts (no source on Etherscan/Sourcify) cannot be fully analyzed; the platform will flag this absence as a High-severity risk signal and prompt the submitter to upload the ABI manually. Bytecode-only proxy detection is limited to known selector and prefix patterns—novel or obfuscated proxies may not be detected without verified source.

Backend and API Specification

The backend is a Python 3.11 FastAPI application containerized with Docker and deployed on AWS (ECS Fargate for the API, Lambda for async document workers). Celery with Redis handles the async task queue for document ingestion and LLM inference jobs. PostgreSQL stores project metadata, extracted claims, flagged issues, and report artifacts. All external LLM calls are routed through a LangChain orchestration layer that enforces structured JSON output via Pydantic and retries with Claude 3.5 Sonnet if GPT-4o returns malformed or low-confidence output.

```
# POST /api/v1/projects - submit project for analysis
{
  "project_name": "string",
  "github_url": "string | null",
  "whitepaper_url": "string | null",
  "contract_address": "string | null",
  "chain": "ethereum | base | arbitrum | polygon",
  "website_url": "string | null"
}
# Returns: { "project_id": "uuid", "status": "queued", "estimated_seconds": 60 }

# GET /api/v1/projects/{project_id}/report
# Returns: full RiskReport JSON with scorecard, flags[], citations[]

# GET /api/v1/projects/{project_id}/export
# PDF export endpoint - rate-limited separately (see Security Considerations)
```

PROMPT INJECTION MITIGATION

All user-supplied text (whitepaper content, GitHub README, website copy) is sanitized and passed to the LLM as retrieved context only—never as system prompt instructions—preventing adversarial prompt injection via malicious documentation.

Frontend Integration

The frontend is a Next.js 14 (App Router) application written in TypeScript with TailwindCSS and shadcn/ui components. It provides a project submission form, real-time analysis progress tracker, and an interactive risk report viewer with expandable flag cards linked to source citations. Progress tracking is implemented via HTTP polling of `/api/v1/projects/{id}/status` every 5 seconds. WebSockets or Server-Sent Events (SSE) were considered but deprioritized for the MVP: analysis jobs are coarse-grained (5 discrete pipeline stages, each 15–60 seconds), making sub-second push updates unnecessary and adding infrastructure complexity; this decision should be revisited if UX feedback indicates polling latency is perceptible. Authentication uses NextAuth.js with wallet-connect (MetaMask/WalletConnect) and email/password as fallback for non-crypto users such as VC analysts.

Submission form: URL inputs for GitHub, whitepaper, website, contract address + chain selector

Progress tracker: step-by-step status (Ingesting → Extracting → Analyzing → Scoring → Done) with elapsed time; polling interval 5 seconds

Report viewer: risk scorecard banner (High/Medium/Low), collapsible issue cards with cited excerpts and remediation suggestions

PDF export: server-side rendered via Puppeteer, includes ProofFlow AI branding and disclaimer footer; rate-limited independently of submission quota (see Security Considerations)

Responsive design: optimized for desktop (primary auditor/analyst use case) with mobile-readable report summary

UX PRIORITY

Non-technical investors and launchpad reviewers are the primary report consumers—every flagged issue must include a plain-English explanation alongside the technical citation, with no crypto jargon assumed.

Security Considerations

Three threat classes require explicit mitigation in the MVP: LLM hallucination producing false accusations, prompt injection via malicious document content, and data privacy exposure of confidential whitepapers to third-party LLM APIs. Hallucination is addressed by requiring every LLM output claim to cite a specific retrieved chunk (RAG-grounded) and applying a confidence threshold — flags below 70% confidence are downgraded to 'Advisory' status and marked for human review rather than surfaced as definitive findings. Prompt injection is mitigated by strict input sanitization and enforcing structured JSON output schemas via Pydantic before any result is persisted. For enterprise customers with strict data-residency requirements, ProofFlow AI supports self-hosted on-premise deployment: the recommended configuration is Llama 2 70B (preferred for reasoning quality on contradiction detection tasks; 13B is acceptable for cost-constrained deployments with a ~15–20% expected recall reduction on complex cross-document contradictions), served via vLLM on a single NVIDIA A100 80 GB GPU (minimum 40 GB VRAM for 70B at 4-bit quantization, or 24 GB VRAM for 13B FP16). Ollama is supported as an alternative for smaller deployments. Typical inference latency for a full report is 3–6 minutes on 70B versus 45–90 seconds for GPT-4o API calls; clients should budget accordingly. GPU instance cost is approximately \$800–1,500/month on AWS (p3.2xlarge or equivalent) or GCP.

Data privacy: self-hosted Llama 2 70B (vLLM on A100 80 GB, min 40 GB VRAM at 4-bit quant) or 13B (Ollama, min 24 GB VRAM) for enterprise customers who cannot send documents to OpenAI APIs

Submission rate limiting: all project submission endpoints require JWT authentication; rate-limited to 10 submissions/hour per API key to prevent abuse

PDF export rate limiting: the /export endpoint is rate-limited independently at 20 renders/hour per API key to prevent cost abuse via Puppeteer rendering without triggering the submission quota

Report liability: all generated reports include mandatory disclaimer that findings are AI-generated, non-exhaustive, and do not constitute legal or financial advice

GitHub token scoping: use read-only fine-grained GitHub PATs; never store tokens beyond request lifetime

False positive management: implement a 'dispute' endpoint allowing project teams to flag incorrect findings for human reviewer escalation

LIABILITY CONTROL

ProofFlow AI reports must never be presented as audit certifications. Enforced disclaimers, confidence scoring, and a human-review escalation path for High-severity findings are non-negotiable MVP requirements to manage regulatory and reputational risk.

Testing Strategy

Testing is divided into four layers: unit tests for the extraction and rule engine logic (pytest, targeting 80%+ coverage on core modules), integration tests for each ingestion connector (GitHub API, Etherscan API, PDF parser) using mocked responses and real fixture documents, end-to-end pipeline tests against a curated set of 10 known-good and 10 known-bad historical Web3 projects to validate that documented rug pulls and contradictions are correctly flagged, and LLM output validation tests that assert Pydantic schema conformance and citation presence on every model response.

Unit: pytest + hypothesis for rule engine edge cases; example invariant: for any token allocation input, the sum of all vesting tranches must equal the total supply within $\pm 0.1\%$ tolerance—hypothesis generates random allocation distributions to surface floating-point rounding failures

Integration: VCR.py cassettes to replay Etherscan and GitHub API responses deterministically in CI

E2E benchmark set: 20 historical projects (10 legitimate launches, 10 documented scams/rug pulls) used as ground truth for precision/recall scoring

LLM eval: LangSmith tracing for every GPT-4o call in staging; flag any response missing citation field

Load test: Locust simulating 50 concurrent report submissions to validate Celery queue and Redis throughput

QUALITY GATE

MVP must achieve $\geq 85\%$ recall on the 10 known-scam benchmark set (miss fewer than 2 documented high-risk signals) before public launch—false negatives on high-severity issues are more damaging to credibility than false positives.

Deployment Roadmap

The 10-week MVP targets Ethereum mainnet ABI support only, with multi-chain expansion (Base, Arbitrum, Polygon) in Phase 2. AWS is the primary deployment target: ECS Fargate for the FastAPI backend, RDS PostgreSQL, ElastiCache Redis, and S3 for document storage. GitHub Actions CI/CD pipeline runs lint, unit tests, and integration tests on every pull request with automatic staging deployment on merge to main.

Phase	Weeks	Budget	Deliverables
Phase 1: Foundation	1–2	Within \$40–70k MVP budget	Architecture finalized, LLM provider evaluated, Docker environment, GitHub + Etherscan connectors prototyped
Phase 2: Ingestion Pipeline	3–5	Within \$40–70k MVP budget	PDF + GitHub + ABI ingestion complete, Pinecone vector store populated, structured extraction with Pydantic schemas
Phase 3: Intelligence Layer	6–8	Within \$40–70k MVP budget	Consistency rules engine, LLM contradiction detection, risk scoring aggregator, confidence thresholds implemented
Phase 4: MVP Launch	9–10	Within \$40–70k MVP budget	Next.js report UI, PDF export, authentication, staging QA, benchmark testing, public beta release
Phase 5: Post-MVP Scale	11–16	Budget TBD — outside \$40–70k MVP scope; estimated \$30–60k depending on GPU infrastructure and team continuity	Base/Arbitrum/Polygon ABI support, launchpad API, on-premise Llama 2 option, dispute/human-review workflow

GO-LIVE CONDITION

Phase 4 public beta targets launchpad and auditor pilot users only—not general public—to control report volume, gather structured feedback, and iterate on LLM prompt accuracy before scaling. Phase 5 investment should be scoped separately once pilot revenue and usage data are available.

Cost Estimation

The \$40k–70k budget covers 10 weeks of development across five team members operating at co-founder or deferred-rate compensation, plus direct infrastructure and tooling costs. LLM inference is the largest variable cost: during the MVP phase (low volume, full-length documents, prompt iteration overhead), GPT-4o costs approximately \$1.00–2.00 per report. At production scale (1,000 reports/day), per-report cost is expected to drop to \$0.10–0.50 through prompt optimization, token capping, and caching of repeated ABI lookups—consistent with the research baseline. This projects to \$100–500/day at scale, requiring a per-report price of at least \$5 to maintain margin at the low end. The MVP inference budget of \$300–800 reflects early-phase costs before optimization; the \$0.10–0.50 scale figure is not yet achievable during the 10-week build and should not be used for MVP cost modeling. The on-premise Llama 2 option (70B on A100) eliminates per-call LLM costs but adds GPU infrastructure overhead (~\$800–1,500/month).

Workstream	Low Estimate (USD)	High Estimate (USD)
Team labor (10 weeks, 5 members)	\$32,000	\$56,000
AWS infrastructure (ECS, RDS, Redis, S3)	\$800	\$1,500
LLM API costs (OpenAI/Anthropic, MVP phase at \$1.00–2.00/report)	\$300	\$800
Pinecone vector DB (Starter → Standard tier)	\$200	\$700
Tooling: GitHub Actions, LangSmith, Etherscan API Pro	\$200	\$500
Design assets and PDF rendering (Figma, Puppeteer hosting)	\$150	\$400
Total MVP Budget	\$33,650	\$59,900

BUDGET RISK

At \$40–70k total, team compensation averages \$8–14k per person over 10 weeks—viable only with co-founder equity arrangements or deferred salaries. Exceeding the LLM inference budget is the most likely cost overrun; implement hard per-project token limits (max 200k tokens/report) at launch to cap exposure. At production scale, the target inference cost of \$0.10–0.50/report (\$100–500/day at 1,000 reports/day) is achievable through prompt optimization and caching but should not be assumed during the MVP phase.

